

Exercice 1 (10 points)

Afin de modéliser en java, l’affichage de noms de fichiers et de dossiers, écrivez les classes suivantes :

- 1) L’interface « **FichierAbstrait** » possédant une méthode afficher() ;
- 2) La classe « **Fichier** » implémente «FichierAbstrait» et possède :
 - a. deux attribut privés nom de type String et retrait de type Retrait.
 - b. un constructeur avec deux paramètres String et Retrait
 - c. une méthode afficher(), qui affichera d’abord un retrait puis le nom du fichier
- 3) La classe « **Dossier** » implémente aussi FichierAbstrait et possède :
 - a. deux attribut privés nom de type String et retrait de type Retrait.
 - b. un attribut fichiers de type « ArrayList », pouvant contenir des dossiers et des fichiers.
 - c. un constructeur avec deux paramètres String et Retrait
 - d. une méthode ajouter (...), qui permettra d’ajouter au tableau « fichiers » des fichiers et des dossiers.
 - e. une méthode afficher(), qui affichera d’abord un retrait puis le nom du dossier. Ensuite, elle augmentera le retrait puis elle affichera tous les fichiers et les dossiers du tableau « fichiers ». Enfin elle diminuera le retrait.
- 4) La classe « **Retrait** » qui permettra de gérer le retrait selon le niveau du fichier ou du répertoire.

```
class Retrait {  
    private StringBuffer sbRetrait = new StringBuffer();  
    public String getRetrait () { return sbRetrait.toString(); }  
    public void augmenterRetrait () { sbRetrait.append("___"); }  
    public void diminuerRetrait () {  
        if (sbRetrait.length() >= 3) { sbRetrait.setLength(sbRetrait.length() - 3); }  
    }  
}
```

- 5) La classe principale « **Program** », qui permettra :
 - a. de créer un objet Retrait commun à tous les fichiers et dossiers
 - b. de créer cinq (05) objets « Fichier », et trois objets « Dossier » tels que :

Dossier	Contient
d1	f1
d2	f2, f3, d1
d3	f4, d2, f5

f1, f2, f3, f4 et f5 sont des objets « Fichier » dont les noms sont respectivement :

a, b, c, d et e

d1, d2 et d3 sont des objets « Dossier » dont les noms sont respectivement :

dos1, dos2 et dos3

- c. d’afficher le dossier d3 :

```
dos3  
___d  
___dos2  
___b  
___c  
___dos1  
___a  
___e
```

Exercice 2 (05 points)

Soient les classes suivantes :

```
class Etudiant {  
    private int note ;           // 0 <= note <= 20  
    public Etudiant ( int note ) {  
        this.note = note ;  
    }  
}
```

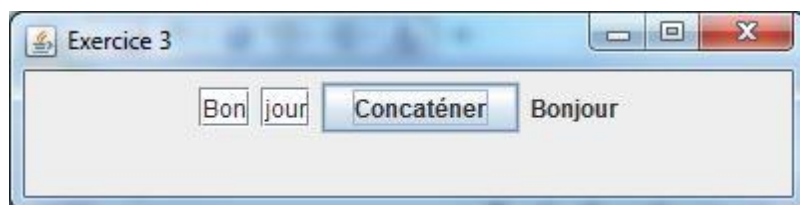
```
class TestNote {  
    public static void main ( String [ ] args ) {  
        int n=0;  
        n = Integer.parseInt(args[0]);  
        Etudiant etudiant = new Etudiant ( n ) ;  
        if (n>=10) System.out.println("Admis(e)");  
        else System.out.println("Ajourné(e)");  
    }  
}
```

Complétez le programme ci-dessus pour que les erreurs susceptibles de se produire soient gérées.

1. **n** n'est pas une note valide (n entier, compris entre 0 et 20):
 - a. Créez la classe "**NoteException**"
 - b. Dans la classe Etudiant, levez (throw) NoteException
 - c. Dans la classe TestNote, lorsque par exemple n=22, attrapez (catch) cette exception et affichez le message suivant : **NoteException: 22 n'est pas une note valide** en redéfinissant toString() dans la classe NoteException.
2. Aussi, gérez l'erreur s'il n'y a pas de paramètre sur la ligne de commande (args[0] n'existe pas) en précisant le type de l'exception et en affichant : **java.lang.ArrayIndexOutOfBoundsException: 0**

Exercice 3 (05 points)

1. Ecrivez une classe **MaFenetre** qui hérite **JFrame** et qui implémente **ActionListener**
 - a. Ajoutez dans cette classe deux (02) **JTextField** qui permettront de saisir du texte
 - b. Ajouter un **JButton** avec l'intitulé "Concaténer"
 - c. Ajoutez un **JLabel** qui permettra l'affichage de la concaténation
 - d. Fixez la taille de la fenêtre, par exemple à (400,100)
 - e. Alignez ces composants avec **new FlowLayout()**
 - f. En cliquant sur le bouton, les deux textes des JTextField seront concaténés et affichés par le JLabel
2. Ecrivez la classe principale qui permettra d'instancier MaFenetre et de l'afficher.



NB. : JTextField est un composant qui permet l'édition de texte. Comme JLabel, le texte peut être modifié par setText() et retourné par getText().